Categories and Search Review and Fix Plan

Date: 2026-04-29 Scope: PSScript hosted Netlify + Supabase app, with emphasis on script categories, category management, and script search/filtering.

Executive Summary

Categories and search should be treated as one discovery system:

- Categories are curated scopes for browsing and filtering.
- Search is text retrieval across title, description, content, tags, and category metadata.
- Filters refine a search result set without changing the user-visible mental model.
- Supabase Postgres should remain the source of truth; no local database should be introduced.

Current code review shows the category settings page is mostly implemented, but the public category browser and search page are not reliably connected to the hosted API behavior. The biggest issue is that the search page sends search and category filters to `GET /api/scripts`, while the hosted Netlify `GET /api/scripts` endpoint currently ignores those filter parameters. The hosted `GET /api/scripts/search` endpoint exists and uses Postgres full-text search, but the frontend search page does not call it.

Research Inputs

- Supabase Full Text Search docs: Supabase recommends Postgres full-text search for database-backed search and shows generated `tsvector` columns plus GIN indexes for faster queries. It also documents `websearch_to_tsquery` for user-facing search syntax and weighted search columns for relevance ranking. Source: https://supabase.com/docs/guides/database/full-text-search
- PostgreSQL text search docs: PostgreSQL documents `websearch_to_tsquery` and `ts_rank_cd`; ranking accounts for lexical, proximity, and structural information. Source: https://www.postgresql.org/docs/current/functions-textsearch.html

- Baymard filter UX research: filter groups should match user mental models, allow useful multi-select where applicable, and keep filtering/category scopes visible near the result set. Source: https://baymard.com/learn/ecommerce-filter-ui
- Baymard search/category scope research: when a search query maps to a category scope, the UI should guide users into that scope instead of leaving category-specific filters unavailable. Source: https://baymard.com/blog/autodirect-searches-matching-category-scopes

Current Implementation Map

Frontend

- `src/frontend/src/pages/Search.tsx`
- Holds `query`, `category`, `tags`, `sortBy`, and `onlyMine`.
- Calls `scriptService.getScripts(queryParams)`.
- Does not call `scriptService.searchScripts()`.

- Displays `script.category?.name`, but does not expose a clear applied-filter summary or category-scope suggestions.

- `src/frontend/src/pages/Categories.tsx`

- Loads categories via `categoryService.getCategories()`.
- Selects a category locally.
- Calls `scriptService.getScripts({ categoryId: selectedCategory.id, limit: 50 })`.

- Uses `categoryId`, while several backend paths expect `category_id` or do not handle the filter at all.

- `src/frontend/src/pages/Settings/CategoriesSettings.tsx`

- Supports create, edit, delete, and inline search.
- Relies on `scriptCount` to choose whether to show the safer "uncategorize and delete" confirmation.

- The hosted `/categories` response currently does not return `scriptCount`, so categories with scripts can appear as if they have zero scripts.

- `src/frontend/src/services/api.ts`

- `scriptService.getScripts()` calls `/scripts`.
- `scriptService.searchScripts()` calls `/scripts/search`.
- `categoryService.getCategories()` calls `/categories`.

Hosted Netlify API

- `netlify/functions/api.ts`
- `handleCategories()` returns `id`, `name`, `description`, `created_at`, and `updated_at`, but not `scriptCount`.
- `handleScripts()` returns recent scripts and ignores query, category, tags, sort, and mine parameters.
- `handleScriptSearch()` uses `websearch_to_tsquery('simple', $2)`, `search_vector`, `ILIKE` fallback, and `ts_rank_cd`, but does not accept category, tag, sort, or only-mine filters.
- `scriptSelect` already joins categories and profiles, so category display data is available.

Database

- `supabase/migrations/20260427_hosted_search_similarity_rls_fixes.sql`
- Adds `scripts.search_vector` as a generated weighted `tsvector`.
- Adds `idx_scripts_search_vector`.
- Adds category and updated-at indexes.
- Adds trigram indexes for title and description.

This is a solid Supabase/Postgres foundation. The main repair is wiring and API contract alignment, not replacing the database design.

Findings

Finding 1 - Search page calls the wrong API path

Evidence:

- `src/frontend/src/pages/Search.tsx` builds `queryParams` with `query`, `category`, `tags`, `sort`, and `mine`.
- It then calls `scriptService.getScripts(queryParams)`.
- `scriptService.getScripts()` sends the request to `/scripts`.

- Hosted `handleScripts()` ignores those parameters and only returns recently updated visible scripts.

Impact:

- Typing in the search box may not change hosted results.
- Applying a category in `/search` may not filter hosted results.
- Sorting controls may appear to work visually while returning the same server order.

Fix:

- Change `/search` page data loading to call `scriptService.searchScripts(filters.query, normalizedFilters)`.
- Normalize frontend filter names to the API contract:
- `q`: user query
- `category_id`: numeric category id
- `tags`: comma-separated tag names or repeated params
- `sort`: `relevance`, `updated`, `created`, `name`, `quality`, `executions`
- `mine`: `true` only when enabled
- Keep `getScripts()` for simple recent scripts lists.

Finding 2 - Hosted search endpoint does not apply category, tag, mine, or sort filters

Evidence:

- `handleScriptSearch()` reads only `q` and `limit`.
- It does not read `category_id`, `tags`, `mine`, or `sort`.

Impact:

- Even after the search page calls `/scripts/search`, filters will still be incomplete unless the endpoint is expanded.
- Category browse and search cannot share one reliable backend contract.

Fix:

- Extend `handleScriptSearch()` to parse:
- `category_id`
- `tags`

- `sort`
- `mine`
- `limit`
- `offset` or `page`
- Build parameterized SQL using a small helper that appends safe where clauses and bind parameters.
- Preserve the current `search_vector` query and `ILIKE` fallback.
- Sort by relevance first only when `q` is present and `sort` is not explicitly set.

Finding 3 - Category browser uses a filter shape the hosted API does not honor

Evidence:

- `src/frontend/src/pages/Categories.tsx` calls `scriptService.getScripts({ categoryId: selectedCategory.id, limit: 50 })`.
- Hosted `handleScripts()` ignores `categoryId`.

Impact:

- Selecting a category can show all scripts instead of only category scripts in the hosted app.

Fix:

- Option A, preferred: have `Categories.tsx` call `scriptService.searchScripts('', { category_id: selectedCategory.id, sort: 'updated', limit: 50 })`.
- Option B: teach `handleScripts()` to support filters too.
- The preferred design is one discovery endpoint for all filtered script retrieval: `/scripts/search`.

Finding 4 - Category settings delete safety depends on missing `scriptCount`

Evidence:

- `CategoriesSettings.tsx` checks `c.scriptCount`.
- `handleCategories()` does not return `scriptCount`.

Impact:

- A category containing scripts can look empty in settings.
- Delete may attempt a direct category delete instead of the safer uncategorize flow.

Fix:

- Update `handleCategories()` to return a script count:
- `COUNT(s.id)::int AS "scriptCount"` with a `LEFT JOIN scripts s ON s.category_id = c.id`.
- Scope counts to scripts visible to the current user if the endpoint becomes authenticated for normal users; for admin settings, global counts are acceptable.
- Keep the existing modal flow for categories with scripts.

Finding 5 - Search result display expects analysis data that the hosted list response does not include

Evidence:

- `Search.tsx` reads `script.analysis?.quality_score`.
- `scriptSelect` does not join the latest `script_analysis`.

Impact:

- Quality sort and quality badges can show `N/A` even when analysis exists.

Fix:

- Either remove quality sort until the API returns analysis quality, or extend `scriptSelect`/search query to join the latest analysis row.
- Best whole-project approach: add a separate `latest_analysis` lateral join to the discovery endpoint only, not to every script detail query.

Target Design

API Contract

Use `/api/scripts/search` as the canonical script discovery endpoint.

Supported query parameters:

Parameter	Type	Behavior
`q`	string	Full-text query; empty means browse/filter mode
`category_id`	number	Restricts results to one category
`tags`	comma-separated string	Restricts results to scripts with selected tags
`mine`	boolean	Restricts results to the current user's scripts
`sort`	enum	`relevance`, `updated`, `created`, `name`, `quality`, `executions`
`limit`	number	1-100
`offset`	number	Pagination offset

Response shape:

```
{
  "scripts": [],
  "total": 0,
  "filters": {
    "q": "",
    "category_id": null,
    "tags": [],
    "mine": false,
    "sort": "updated"
  }
}
```

SQL Design

- Keep `scripts.search_vector` generated column.
- Keep `idx_scripts_search_vector`.
- Use `websearch_to_tsquery('simple', $query)` for forgiving user syntax.
- Use `ts_rank_cd(search_vector, tsq)` for relevance when `q` exists.
- Add category filter with `s.category_id = $n`.
- Add tags filter with `EXISTS` over `script_tags` and `tags`.

- Add quality sort only if the endpoint joins latest analysis.
- Return `total` via a separate count query or `COUNT(*) OVER()` if the SQL stays readable.

UI Design

- Search box should be the primary control.
- Category filter should be visible as a facet, not hidden behind settings.
- Applied filters should appear as removable chips: query, category, tags, mine.
- Empty states should say exactly which filters produced no results and offer "Clear filters".
- Category browse should reuse the same result card/list component as search.
- Category settings should remain an admin-oriented CRUD view, separate from discovery.

Implementation Plan

1. Fix hosted category payload.
2. Add `scriptCount` to `GET /api/categories`.
3. Keep category CRUD permissions unchanged.

4. Verify: category settings displays nonzero counts for categories with scripts.

5. Create a shared search parameter normalizer.

6. Convert `category` / `categoryId` to `category_id`.
7. Convert `tags` to a consistent array.
8. Clamp `limit` and `offset`.
9. Validate `sort` against an allowlist.

10. Verify: unit tests for each query shape.

11. Expand `handleScriptSearch()`.

12. Support `q`, `category_id`, `tags`, `mine`, `sort`, `limit`, and `offset`.
13. Preserve current FTS plus ILIKE fallback.
14. Add count/pagination metadata.

15. Verify: hosted API tests assert SQL contains indexed `search_vector` path and category/tag filters.

16. Wire `/search` page to `scriptService.searchScripts()`.

17. Keep React Query key stable and serializable.
18. Debounce typing or search on submit, but do not rely on a manual `refetch()` against stale params.

19. Verify: typing query, selecting category, selecting tags, and reset each change visible results.

20. Wire `/categories` page to `/scripts/search`.

21. Pass an empty query with `category_id`.
22. Preserve selected category in the URL.

23. Verify: selecting a category returns only scripts in that category.

24. Make result display consistent.

25. Prefer one `ScriptSearchResultCard` or reuse `ScriptCard`.
26. Show category name, updated date, owner, version, and analysis score only when provided.

27. Verify: no `undefined`, bad dates, or misleading `N/A` in normal hosted data.

28. Add regression tests.

29. Frontend tests for Search query construction.
30. Frontend tests for Categories page category filter construction.
31. Netlify API tests or static contract tests for `/scripts/search`.
32. Browser smoke test:
    - Login.
    - Upload a script with a known title and category.
    - Search by title.
    - Filter by category.
    - Clear filters.
    - Delete the test script.

Test Matrix

Area	Test	Expected Result
Category payload	`GET /api/categories`	Every row includes `scriptCount`
Search text	`q=Get-Process`	Matching script title/content returned first
Empty browse	no `q`, no filters	Recent visible scripts returned
Category filter	`category_id=1`	Only scripts in category 1 returned
Query + category	`q=process&category_id=1`	Intersection of search and category
Tags	`tags=cleanup,windows`	Only scripts with matching tags returned
Mine	`mine=true`	Only current user's scripts returned
Sort updated	`sort=updated`	Updated descending
Sort relevance	`q=process&sort=relevance`	Ranked by `ts_rank_cd`, then updated
Delete category with scripts	category count > 0	Confirmation uses uncategorize path

Risks and Controls

- Risk: dynamic SQL can become unsafe.
- Control: never interpolate user values; append only allowlisted SQL fragments and bind all user values.
- Risk: search performance regresses.
- Control: keep `search_vector` generated column and GIN index; use trigram indexes only for fallback matching.
- Risk: UI state and server state drift.
- Control: make `/scripts/search` the single discovery endpoint for search and category browsing.
- Risk: category deletion hides data.

- Control: keep the existing uncategorize behavior; do not delete scripts when deleting a category.

Recommended Fix Order

1. Backend `/categories` count and `/scripts/search` filter support.
2. Frontend `/search` endpoint wiring.
3. Frontend `/categories` endpoint wiring.
4. Display cleanup and reusable result cards.
5. Regression tests and browser smoke test.

This order fixes correctness before UI polish and keeps the hosted Supabase database as the single data source.

Generated 2026-04-29 14:08 UTC